Push your changes

Run checks, stage reviewed files, then push main.

01 Run checks

Open PowerShell in the project root and run the checks before publishing.

```
git status --short
git diff --check

Set-Location api
npm test
npm run test:types
npm run build

Set-Location ..\web
npm run build

Set-Location ..
```

Do not open, print, stage, or commit .env. It contains private configuration and should remain ignored by Git.

Stage, review, and push

Commit only the intended changes to the configured GitHub origin.

02 Stage and review

This file list matches the current changes. Adjust it if your git status is different.

```
git add README.md SRS.md api/src/authorization.ts ‘
  api/src/contractors.controller.ts api/src/database.service.ts ‘
  api/src/jobs.controller.ts api/src/organization-locations.controller.ts ‘
  api/src/organization-settings.controller.ts api/src/permissions.ts ‘
  api/tests/jobs.controller.test.ts ‘
  api/tests/organization-settings.controller.test.ts ‘
  api/tests/permissions.test.ts ‘
  web/src/App.tsx web/src/print.css web/src/theme.css

git diff --cached --check
git diff --cached --name-only
```

Review the staged filenames and changes. Confirm .env is not listed before committing.

03 Commit and push

```
git commit -m "Improve work order scheduling and printing" ‘
  -m "Co-authored-by: Copilot <223556219+Copilot@users.noreply.github.com>"

git push origin main
```

If git reports there is nothing to commit, inspect git status before continuing.

Deploy the frontend

Cloudflare Pages builds the static React app from GitHub main.

01 Automatic deployment

If Cloudflare Pages is connected to this GitHub repository, pushing to main starts a production deployment. Open the Pages project and check the latest deployment status and build log.

02 Confirm build settings

- Production branch: main
- Root directory: web
- Build command: npm run build
- Build output directory: dist
- Node.js: 22

03 Configure the API URL

In Pages project settings, add the following build variable. Redeploy after changing it.

```
VITE_API_BASE_URL=https://your-public-api.example.com
```

- Use the public HTTPS API origin only; no /api/v1 path and no trailing slash.
- This value is embedded in the public frontend bundle. Never put passwords, tokens, or other secrets in it.

> **An HTTPS Pages site cannot safely call an HTTP API. The API must be publicly reachable over HTTPS.**

04 Configure CORS

After Cloudflare provides the Pages URL, set CORS_ORIGIN on the API server to that exact origin, such as https://your-project.pages.dev. Restart or redeploy the API after changing it.

> **Keep PostgreSQL private. Do not expose port 5432 to the public internet.**

Deploy the backend

The API and PostgreSQL run on the backend server using Docker Compose.

01 Update the server checkout

Connect to the backend server and open PowerShell in the project directory. Keep the server production .env in place; do not copy your local .env over it.

```
git pull origin main
```

02 Rebuild and restart

```
docker compose -f compose.yaml up --build -d
docker compose -f compose.yaml ps
```

Compose rebuilds the app image and starts services in the background. PostgreSQL data is stored in the pgdata volume.

03 Check health

Open the health endpoint through the public HTTPS API origin:

```
https://your-public-api.example.com/api/v1/health
```

- Confirm the API responds successfully.
- Confirm the database service is healthy in docker compose ps.

Protect production data

Never run docker compose down -v during an update. The -v option removes volumes and can delete PostgreSQL data.

Do not expose PostgreSQL port 5432 publicly. Use strong production credentials and JWT secrets on the server.

Verify the deployment

Test production after the frontend and backend are both updated.

01 Smoke-test the app

- Open the Cloudflare Pages site and sign in.
- Create a test work order; confirm its success message and job reference.
- Open the work order, set its Planned work date, and save.
- Open Schedule and verify it appears in the selected week.
- Print a work order and check the suggested PDF name includes its number.

If frontend cannot reach the API

- Check VITE_API_BASE_URL is the correct HTTPS API origin, with no path or trailing slash.
- Check CORS_ORIGIN exactly matches the deployed Pages origin, including https://.
- Redeploy Pages after changing a frontend build variable.
- Rebuild/restart the API after changing CORS_ORIGIN or backend code.
- Confirm the API health endpoint responds over HTTPS.

Deployment split

Cloudflare Pages hosts only the static frontend. Keep the API and PostgreSQL running on the backend server. A push can trigger the Pages build automatically, but backend deployment is a separate step.

See README.md for repository deployment notes. Dockerfile builds the container; compose.yaml configures the local/server stack.